# Help Desk Management System — Execution Roadmap & Repo Structure

*Rebuild of PowerApps HDMS → HTML/CSS/JS + PHP + MySQL, team build via GitHub*

---

## 0. How to use this document

This is your team's single source of truth. Print it, pin it in your group chat, whatever — but every person on the team should know which phase you're in and what "done" looks like for that phase before moving on. Don't let anyone jump ahead to Phase 3 features while Phase 1 screens are still half-converted; that's how projects rot.

**Suggested role split** (adjust to your actual team size):

- **Frontend Lead** — owns screen conversion, CSS system, JS-DOM wiring
- **Backend Lead** — owns PHP API endpoints, auth, business logic
- **DB/Integration Lead** — owns schema, `schema.sql`, connects frontend calls to backend responses
- **QA/PM** — owns GitHub board, testing checklist, keeps scope honest

If you're 2-3 people, double up roles. If it's just you and one other person, Frontend+Backend can be one person, DB+QA the other.

---

## 1. Full Repository / Folder Structure

```
help-desk-system/
│
├── frontend/
│   ├── assets/
│   │   ├── css/
│   │   │   ├── variables.css          # colors,
spacing, font tokens (design system)
│   │   │   ├── style.css              # global
resets, typography, layout
│   │   │   ├── components.css         # buttons,
cards, modals, tables, badges
│   │   │   └── auth.css               #
login/register page specific styles
│   │   ├── js/
│   │   │   ├── api.js                 # fetch()
wrapper for all backend calls
│   │   │   ├── auth.js                #
login/logout/session check logic
│   │   │   ├── tickets.js             # ticket
CRUD + status updates
│   │   │   ├── assets-module.js       # asset
register logic
│   │   │   ├── server-access.js       # access
log logic
│   │   │   ├── documentation.js       # doc repo
upload/search
│   │   │   ├── monitoring.js          # stretch:
monitoring dashboard
│   │   │   └── utils.js               # date
formatting, form validation, toasts
```

```
│   │   └── images/
│   │       ├── logo.png
│   │       └── icons/
│   │
│   ├── pages/
│   │   ├── auth/
│   │   │   ├── login.html
│   │   │   ├── register.html          # if self-
registration is allowed
│   │   │   └── forgot-password.html
│   │   ├── dashboard/
│   │   │   ├── admin-dashboard.html
│   │   │   ├── ict-staff-dashboard.html
│   │   │   └── enduser-dashboard.html
│   │   ├── tickets/
│   │   │   ├── ticket-list.html
│   │   │   ├── ticket-create.html
│   │   │   └── ticket-detail.html     # includes
status history/comments
│   │   ├── server-access/
│   │   │   ├── access-log-list.html
│   │   │   └── access-log-entry.html  # security
check-in/out form
│   │   ├── assets/
│   │   │   ├── asset-register.html
│   │   │   ├── asset-detail.html
│   │   │   └── asset-checkin-checkout.html
│   │   ├── documentation/
│   │   │   ├── doc-repository.html
│   │   │   └── doc-upload.html
│   │   ├── monitoring/                # stretch
goal, build last
│   │   │   └── monitoring-dashboard.html
│   │   └── admin/
│   │       ├── user-management.html
│   │       ├── role-management.html
│   │       └── reports.html
│   │
│   └── partials/
│       ├── navbar.html                # or
```

```
navbar.php if using PHP includes
│       ├── sidebar.html
│       └── footer.html
│
├── backend/
│   ├── api/
│   │   ├── auth/
│   │   │   ├── login.php
│   │   │   ├── logout.php
│   │   │   ├── register.php
│   │   │   └── session-check.php
│   │   ├── tickets/
│   │   │   ├── create.php
│   │   │   ├── list.php
│   │   │   ├── update-status.php
│   │   │   ├── assign.php
│   │   │   └── delete.php
│   │   ├── server-access/
│   │   │   ├── log-entry.php
│   │   │   ├── log-exit.php
│   │   │   └── list.php
│   │   ├── assets/
│   │   │   ├── create.php
│   │   │   ├── list.php
│   │   │   ├── update.php
│   │   │   ├── checkin.php
│   │   │   ├── checkout.php
│   │   │   └── qr-generate.php
│   │   ├── documentation/
│   │   │   ├── upload.php
│   │   │   ├── list.php
│   │   │   └── delete.php
│   │   ├── users/
│   │   │   ├── create.php
│   │   │   ├── list.php
│   │   │   └── update-role.php
│   │   └── monitoring/                # stretch
goal
│   │       ├── ping-check.php
│   │       └── report.php
```

```
│   ├── config/
│   │   ├── db.php                      # PDO
connection, one source of truth
│   │   └── constants.php               # role IDs,
status enums, etc.
│   └── includes/
│       ├── auth-middleware.php         # checks
session + role before allowing access
│       ├── functions.php               # sanitize
input, JSON response formatter
│       └── mailer.php                  # email/SMS
notification logic
│
├── database/
│   ├── schema.sql                      # full
CREATE TABLE statements
│   ├── seed-data.sql                   #
sample/demo data for testing
│   └── ERD.png                         # exported
from dbdiagram.io
│
├── docs/
│   ├── BRD.pdf
│   ├── screen-inventory.xlsx           # every
PowerApps screen, mapped to new page
│   ├── api-documentation.md            # endpoint
list, request/response formats
│   └── setup-guide.md                  # how to
run this locally (XAMPP/Laragon steps)
│
├── .gitignore                          # ignore
/config secrets, node_modules if any, .DS_Store
└── README.md
```

**Pro tip:** create every empty folder with a placeholder `.gitkeep` file so Git actually tracks the structure from day one — Git doesn't track empty folders.

---

## 2. Detailed Phase-by-Phase Roadmap

### Phase 0 — Foundation (Days 1–3)

| Task | Owner | Output |
|---|---|---|
| Create GitHub repo + folder skeleton above | DB/Integration Lead | Repo pushed, structure visible |
| Set up branch protection on `main` (require PR) | PM/QA | Branch rules active |
| Everyone installs XAMPP/Laragon locally | All | Local PHP+MySQL running |
| Create GitHub Projects board with columns: Backlog / To Do / In Progress / Review / Done | PM/QA | Board live |
| Screen inventory: list every PowerApps screen into `docs/screen-inventory.xlsx` | Frontend Lead | Complete list w/ screen name → new page mapping |
| Draft initial ERD on dbdiagram.io from the module list | DB Lead | `ERD.png` in `/database` |

**Definition of Done for Phase 0:** everyone can clone the repo, run it locally (even if empty), and see the same folder structure. No code written yet — this is pure setup.

**Common mistake:** skipping the screen inventory and just "winging it" screen by screen. You will duplicate work and forget screens exist. Do the spreadsheet.

---

## Phase 1 — Static Frontend Conversion (Days 4–14, ~2 weeks)

Goal: every screen exists as a static HTML page, styled, with placeholder/dummy data. No PHP yet.

**Day-by-day flow (repeat this loop per screen):**

1. Screenshot the PowerApps screen.
2. Feed to AI with a consistent prompt template (see below).
3. Drop output into the correct `frontend/pages/<module>/` file.
4. Wire it to `variables.css` (already-defined colors/fonts) so it matches the rest of the app, not a random AI-generated palette.
5. Add `<!-- TODO: dynamic data from backend -->` comments where lists/tables will later populate from PHP.
6. Commit on a feature branch: `feature/frontend-tickets-screens`, `feature/frontend-assets-screens`, etc.
7. Open PR into `dev`, one teammate reviews for visual consistency, merge.

**Standard AI prompt template to reuse for every screen:**

> "Here's a screenshot of a PowerApps screen called [screen name]. Convert it to a static HTML5 page using our existing CSS classes: [paste your `variables.css` + `components.css` class names]. Keep the layout and field structure as close to the original as possible. Leave `<!-- TODO -->` comments anywhere data would normally load dynamically (tables, dropdowns from a database, user info)."

**Split by module across the week:**

- Days 4–6: Auth screens (login/register) + Dashboard screens
- Days 7–10: Ticketing module screens (list/create/detail)
- Days 11–12: Server access + Asset register screens
- Days 13–14: Documentation repo + Admin screens

**Definition of Done:** every screen in `screen-inventory.xlsx` has a matching `.html` file, styled consistently, navigable via the navbar/sidebar (even with dead links/dummy data).

**Pro tip:** build `navbar.html` and `sidebar.html` ONCE and reference them consistently (or use PHP `include()` from the start) — don't copy-paste nav markup into 15 files. You'll hate yourself when the menu structure changes.

---

## Phase 2 — Database + Core Backend (Days 15–21, 1 week)

Goal: real database exists, PHP can talk to it, one full module (Tickets) works end-to-end.

| Day | Task |
| --- | --- |
| 15 | Finalize ERD → write `schema.sql` with all tables (users, roles, tickets, ticket_updates, server_access_logs, assets, asset_movement_logs, documents) |
| 16 | Run schema locally, write `seed-data.sql` with dummy test records |
| 17 | Build `config/db.php` (PDO connection) + `includes/functions.php` (JSON response helper, input sanitizer) |
| 18–19 | Build auth: `login.php`, `session-check.php`, `logout.php`, plus `auth-middleware.php` for role checks |
| 20–21 | Build full Tickets API: `create.php`, `list.php`, `update-status.php`, `assign.php` — test each with Postman/Thunder Client before touching frontend |

**Definition of Done:** you can log in via a real database user, and hit `tickets/list.php` in a browser or Postman and get back real JSON from MySQL.

**Common mistake:** building all 6 modules' backend at once before confirming ONE module works end-to-end (DB → API → frontend). Prove the pattern with Tickets first, then replicate it — don't parallelize the pattern-proving step.

---

### Phase 3 — Integration: Wire Frontend to Backend (Days 22–35, 2 weeks)

Goal: static pages become live pages, module by module.

**Per-module loop (repeat for each module):**

1. Backend Lead finishes that module's API endpoints.
2. Frontend Lead replaces dummy `<!-- TODO -->` sections with `fetch()` calls via `api.js`.
3. Test the full loop: create a ticket in the UI → confirm it lands in MySQL → refresh page → confirm it displays.
4. QA checks role-based access (does an End User see the Admin panel? They shouldn't).

**Suggested order** (build confidence with the core feature first, expand outward):

- Days 22–26: Tickets module fully live (create, list, update status, assign)
- Days 27–29: Auth + role-based dashboards fully live
- Days 30–32: Asset Register + Server Access Logs live
- Days 33–35: Documentation repo live (file upload/download)

**Definition of Done:** a user can log in, and every core module (tickets, assets, access logs, docs) reads/writes real data from MySQL through the UI.

---

### Phase 4 — Stretch Features (Days 36–42, only if ahead of schedule)

- Simple rule-based chatbot for common ticket categories (NOT full AI/NLP — a keyword-matcher that suggests FAQ

answers before a user submits a ticket is realistic and still impressive)

- Basic monitoring dashboard: a PHP cron script that pings server/network status every few minutes and logs uptime — not real "real-time," but close enough for a demo
- Email/SMS notifications on ticket status change (use PHPMailer + Gmail SMTP, or Africa's Talking API for SMS if you want it Kenya-relevant)

**Don't start this phase until Phase 3's Definition of Done is fully met.** Scope creep here is what kills student team projects in week 6.

---

### Phase 5 — Testing, Hardening & Deployment (Days 43–49, 1 week)

| Day | Task |
|---|---|
| 43 | Full team walkthrough — every module, every role, click everything |
| 44 | Fix cross-browser issues (test Chrome + at least one other) |
| 45 | Security pass: confirm all queries use PDO prepared statements, passwords are hashed (`password_hash()`), sessions expire, file uploads are validated |
| 46 | Pick hosting — Truehost/Hostinger (Kenyan, cheap, cPanel+MySQL included) or InfinityFree for a free pilot |

| Day | Task |
|---|---|
| 47–48 | Deploy, migrate schema to live DB, test live version end-to-end |
| 49 | Prep demo data + walkthrough script for your presentation/viva |

**Definition of Done:** live, hosted version works for all four user roles from the BRD (ICT Staff, End Users, Security, Admin), with realistic seed data loaded.

---

## 3. Git Workflow Cheat Sheet

```
main         → always demo-ready
dev          → integration branch, merge here first
feature/xyz  → one branch per module or screen batch
```

**Daily habit:** pull `dev` before starting work, branch off it, commit small and often (not one giant commit at the end of the week), push, open PR, get at least one review before merging into `dev`. Merge `dev` → `main` only at the end of each phase, after the Definition of Done is met.

**Commit message convention** (keeps history readable):

```
feat(tickets): add ticket status update endpoint
fix(auth): session not persisting after login
style(css): unify button styles across modules
```

---

## 4. Weekly Checkpoint Ritual

Every week, whole team does a 15-minute sync (even over WhatsApp call):

1. What got merged into `dev` this week?
2. What's blocked?
3. Are we still on the roadmap, or has scope crept?
4. Update the GitHub Project board to reflect reality, not wishful thinking.

This alone prevents the classic "we thought someone else was doing that" disaster two days before submission.